# A Mathematical Framework for Building Agent Systems

## Task-First Design, Equivalence, and Model Choice under Deployment Budgets

**Pierfrancesco Beneventano** and **Tomaso Poggio**
*Center for Brains, Minds and Machines, MIT*

**Editor:** Under review

## Abstract

A builder of LLM agent systems faces a concrete design problem: given a task distribution and a compute budget, which model, orchestration, and tool configuration minimizes time to verified solution? We provide a mathematical framework that replaces benchmark racing—the standard methodology of running both models and picking the winner—with structured analysis.

The framework rests on three moves. First, we separate the *task* (an objective under budgets with an external verifier) from the *graph* (the solver-induced execution), and show this distinction changes design decisions. Second, we cast agent building as a stochastic optimization problem over a six-dimensional design tuple $(M, \pi, \Omega, W, K, p)$ and prove that restricting optimization to model variation alone is suboptimal whenever optimal routing policies differ across models. Third, we define two notions of task equivalence—post-training equivalence (when does training data transfer?) and orchestration equivalence (when does the same workflow apply?)—and show they are independent.

For *packed latent-mode families*, expected certified log-speedup decomposes exactly as $\Delta = \log(\kappa_0/\kappa) + \varphi + G - \varepsilon$, separating per-step cost, within-mode competence, task-relevant information, and routing mismatch into four independently measurable terms. This yields a closed-form crossover formula: a smaller specialist dominates a frontier generalist whenever the effective mode count falls below a computable threshold. The decomposition is exact in the packed-family regime and degrades gracefully outside it, with remainder $O(\delta \log M_n + \eta)$. We apply the framework to building agents for formal mathematical research, deriving opposing prescriptions for different task regimes from the same theory, and provide a measurement protocol for estimating all quantities from a structured pilot.

**Keywords:** agent design, certified time, task equivalence, orchestration equivalence, model choice, operations research

## 1 Introduction

A builder of agent systems faces a question every deployment cycle: *I have a task family, a compute budget, and a menu of models ranging from fine-tuned 7B specialists to frontier generalists. What system should I build?* The question is not merely which model to call. It is which model, which orchestration policy, which tools, which memory architecture, and which budget allocation jointly minimize time to verified solution.

The dominant methodology is benchmark racing: run both models on a test set, pick the winner, repeat when the next model arrives. Benchmark racing is fast and requires no

theory. It is also structurally blind. It conflates per-step cost, within-mode competence, task-relevant information, and routing quality into a single opaque comparison. A team that benchmark-races cannot diagnose *why* model A beats model B, cannot predict *when* the ranking reverses on a different task distribution, and cannot decide whether to fine-tune, add tools, or restructure the workflow. Benchmark racing tells you the average winner. This paper tells you the winner *per task regime* and tells you *why*.

**Four questions.** The paper is organized around four questions, ordered so that each builds on the previous and the climax comes last.

1. **What explains suboptimal performance?** When an agent system is slow, is it because the task lacks exploitable structure, because the system routes compute to the wrong strategies, because each step is too expensive, or because the workflow graph is badly shaped? The four-term decomposition (Section 6) separates these four causes into independently measurable channels.

2. **When are two tasks the same for design purposes?** Benchmark racing treats every new task as an independent evaluation problem. Post-training equivalence and orchestration equivalence (Section 5) reveal when two apparently different tasks demand the same system design—and when they do not.

3. **When does training on one task help another?** The transfer characterization (Section 5) bounds the gain from source-task data by the mutual information between the source representation and the target's latent modes. Conditional guidance tells the builder: shared modes implies transfer works; shared modes but different orchestration implies retrain the router; no shared modes implies broader pre-training.

4. **Which model should I use—and when does a small specialist beat a frontier generalist?** The crossover formula (Section 7) gives a closed-form threshold. A smaller model wins when its cost advantage exceeds the information and routing penalties, with the boundary determined by the task's effective mode count.

**Contributions.** The paper makes five contributions.

1. **A task-first ontology for agent systems** (Section 3). We formalize budgeted transductive tasks, stochastic solvers, and certified time, separating the task (objective under budgets) from the graph (solver-induced execution). A formal proposition shows this separation changes design decisions: benchmark racing is suboptimal whenever optimal routing differs across models.

2. **Agent building as a deployment optimization problem** (Section 4). We cast agent design as stochastic optimization over a six-dimensional design tuple and prove that restricting the program to model variation alone forfeits routing-adaptive designs.

3. **Task equivalence and transfer** (Section 5). We define post-training equivalence and orchestration equivalence, prove they are independent, and characterize transfer gain. The orchestration bound is $O(\sqrt{\alpha}\log M)$ via Pinsker and Audenaert.

4. **An exact decomposition with approximation guarantee** (Section 6). The four-term decomposition is algebraically exact in the packed-family regime; outside it, a

bounded remainder holds. The decomposition yields a crossover formula, a specialization-dominance checklist, and a multi-agent threshold (Section 7).

5. **A measurement protocol and application** (Sections 8–9). We instantiate the framework on formal mathematical research, deriving opposing prescriptions for different task regimes, and provide estimators for all framework quantities from a structured pilot.

**Scope.** The main body concentrates on tasks for which success is externally checkable: code against test suites, proofs against proof assistants, plans against constraint checkers. This is a deliberate restriction, not an apology. It is precisely the regime in which time-to-solution becomes mathematically clean and engineering-relevant. The exact decomposition requires the packed latent-mode family (Assumption 6): each instance has a dominant solution strategy, and certified time scales inversely with the compute share allocated to that strategy. Outside this regime, the approximation guarantee (Proposition 25) quantifies the degradation. Appendix A extends the framework to continuous rewards.

**Relation to Achille–Soatto.** Achille and Soatto [1] establish that the mutual information between an agent's history and its decisions bounds expected speedup over random search—formally, that information buys speed. Their framework is model-independent: it characterizes a task's information structure without reference to which model processes it. The moment one asks "which model should I use?", one needs per-step cost and within-mode competence—terms their framework does not contain. We extend their perspective: *speed has a price, the price depends on design choices, and the price structure is decomposable into four measurable channels.* They diagnose how much structure a task contains. We tell you which system to build to exploit that structure.

**Roadmap.** Section 2 positions the paper. Section 3 defines the task, solver, and certified-time objects. Section 4 formulates the deployment optimization problem. Section 5 develops task equivalence and transfer. Section 6 contains the decomposition core. Sections 7–9 derive consequences, apply the framework to mathematical research, and specify the measurement protocol. Section 10 discusses limitations and open problems.

## 2 Related Work and Positioning

**Achille–Soatto and the time-centric transductive view.** The conceptual starting point is Achille–Soatto [1], who frame agents in the verifiable transductive regime and make time-to-solution central. We inherit that move almost entirely. Our contribution is narrower and more operational: we specialize the viewpoint to the problem of *building agent systems*, with explicit objects for tasks, solvers, budgets, transfer, orchestration mismatch, and deployment-facing model choice. The older lineage behind this viewpoint runs through Solomonoff induction, Levin search, and related resource-bounded views of inference [37, 24, 29, 25, 34]. These works motivate the idea that prior weight, description length, and reusable structure can be traded against search time. The present paper stays within that lineage but asks a deployment-facing question: given a modern agent stack with LLMs, tools, memory, and orchestration, which quantities should a builder optimize?

**Graph-centric reasoning and workflow literature.** A substantial body of recent work describes reasoning or orchestration through explicit graph-like objects: trees of thoughts,

graphs of thoughts, and code-represented workflows searched over graph-structured execution spaces [44, 6, 45]. We agree that graphs are often the right representation of a solver execution. Our difference is ontological rather than adversarial: the graph is not the task. The task is the objective under budgets; the graph is the realization induced by a particular solver, model, and memory regime. That separation is what lets us ask whether different solvers on the same task should induce different graphs, and whether apparently similar graphs correspond to the same task at all.

**Algorithm portfolios and anytime computation.** The question "how should one allocate limited compute across competing strategies for a given instance?" has a long history in algorithm portfolios and per-instance selection [20, 43], anytime algorithms and metareasoning under deadlines [47, 19, 33], and sequential inspection [42, 40, 10]. Weitzman's Pandora's Box result gives optimal stopping rules for inspecting alternatives with known reward distributions—structurally close to our routing problem, but in a static setting without the learning-from-verification feedback that the packed-family model captures. The present framework brings this family of ideas into the language of modern agent systems: the builder allocates compute across branches, tools, and strategies, and the target is a certified time frontier rather than a scalar accuracy metric.

**Operations research and team decision problems.** Radner [31] initiated the theory of team decisions: agents with a common objective but decentralized information. Our multi-agent threshold (Section 7) is a special case of Marschak–Radner team optimization [28] applied to the packed-family setting. The broader framing of agent building as a stochastic optimization over design variables follows the tradition of stochastic programming [7] and automated ML pipelines [21, 16]. The contribution is not the optimization framing—that is standard—but the decomposable objective with four named, measurable terms.

**Conditional computation and routing.** Mixture-of-experts and conditional computation architectures separate a predictor from a router and make routing quality a first-class determinant of performance [22, 35, 14]. This is structurally close to our mismatch accounting. The difference is granularity: in this paper the objects being routed are not token-level experts but agentic modes—toolchains, decomposition choices, branches of search, and orchestration policies.

**Task equivalence and MDP similarity.** Ferns, Panangaden, and Precup [15] define bisimulation metrics for MDPs; Ravindran and Barto [32] introduce MDP homomorphisms. Both operate at a finer grain than our equivalence notions: they compare state-action spaces, while we compare posterior mode distributions at the task-family level. Agarwal et al. [2] propose policy similarity metrics. Taylor and Stone [39] survey transfer learning in RL. Our post-training equivalence addresses a different question: not whether two MDPs admit the same optimal policy, but whether training data from one task family improves certified time on another. Orchestration equivalence is, to our knowledge, without close precedent at this grain size.

**Verifiable reasoning and test-time compute.** The motivating workloads are those in which test-time reasoning is evaluated against an external checker: code generation against unit tests, formal theorem proving, and verifier-backed reasoning [9, 27, 46]. A parallel

literature studies how additional test-time compute improves performance [41, 44, 36]. Our contribution is not another prompting or search heuristic. It is a mathematical language for asking what kind of speedup a design has bought and whether that speedup came from more reusable structure, better orchestration, or simply more expensive per-step computation.

**Memory, retrieval, and mode discovery.** Retrieval-augmented models and library-induction systems convert past experience into faster future execution [26, 8, 13, 18]. In our language, such mechanisms change both how much reusable structure is available and how effectively the solver deploys it. Mode discovery—identifying the latent strategies that govern performance—connects to the options framework [38], option-critic architectures [4], and trajectory-based behavioral mode discovery [5]. Our measurement protocol (Section 9) adapts trajectory clustering to the proof-strategy setting.

**Positioning.** We do not claim priority on time, verifiability, search, routing, or graph-based workflows in isolation. The claim is that, within one framework, we separate tasks from solver-induced graphs, cast agent building as a budgeted design problem, formalize task equivalence and transfer, and connect these objects to theorem-level speed accounting and deployment-facing model choice. The paper is a framework for *building* agent systems, not only for interpreting them after the fact.

## 3 A Mathematical Framework for Agentic Tasks

The mathematical objects in this paper are tasks, solvers, and certified time. The definitions are compressed; a Lean 4 tactic-prediction example follows each one to build intuition.

### 3.1 Tasks

**Definition 1 (Budgeted transductive task)** *A* budgeted transductive task *at scale $n$ is a tuple $\mathfrak{T} = (\mathcal{X}_n, \mathcal{Y}_n, V_n, \mathcal{B}_n)$ where:*

- *$\mathcal{X}_n$ is a set of instances and $\mathcal{Y}_n$ a set of candidate solutions;*
- *$V_n : \mathcal{X}_n \times \mathcal{Y}_n \to \{0, 1\}$ is a polynomial-time verifier;*
- *$\mathcal{B}_n = (b_t, b_c, b_m)$ specifies budgets for time, cost, and memory.*

*Success on instance $x$ means producing $y$ with $V_n(x, y) = 1$ within $\mathcal{B}_n$.*

In Lean 4: $\mathcal{X}_n$ is a set of lemma statements, $\mathcal{Y}_n$ is the set of tactic proof scripts, $V_n$ is the Lean kernel checker, and $\mathcal{B}_n$ specifies maximum API cost and wall-clock time.

**Definition 2 (Task family)** *A* task family *is a sequence $\mathfrak{T} = (\mathfrak{T}_n)_{n\geq 1}$ of budgeted transductive tasks, together with a deployment distribution $\mu_n$ over instances $\mathcal{X}_n$. The family captures a class of problems the builder expects to encounter repeatedly.*

The Lean example: a task family is a distribution over Mathlib lemma difficulties—some routine, some library-heavy, some creative.

### 3.2 Solvers and orchestration graphs

**Definition 3 (Stochastic solver)** *A* stochastic solver *$A$ for task family $\mathfrak{T}$ is a randomized algorithm that, given instance $x \in \mathcal{X}_n$ and budgets $\mathcal{B}_n$, produces a candidate $y$ (or reports failure) using a sequence of model calls, tool invocations, memory accesses, and branching decisions. A run of $A$ on $(x, r)$ (where $r$ denotes randomness) induces a* run graph *$G_A(x; r)$—a DAG whose nodes are computational steps and whose edges encode data and control flow.*

The graph is not the task. The task is the objective under budgets; the graph is the realized execution induced by a particular solver, model, and orchestration on a particular instance. Two solvers for the same task may produce radically different graphs—one a linear chain of rewrites, another a branching tree search with backtracking—yet both target the same objective. Confusing the graph with the task is the structural error that benchmark racing commits: it evaluates the realized execution without decomposing the objective.

### 3.3 Certified time

**Definition 4 (Certified time-to-solution)** *For solver $A$ on task family $\mathfrak{T}$ at confidence level $\delta \in (0, 1)$, the* certified time-to-solution *is*

$$\overline{T}_A(n, \delta) \;=\; \inf\{t > 0 : \mathbb{P}_{x \sim \mu_n, r}[A(x; r) \text{ succeeds within time } t] \geq 1 - \delta\}\,.$$

Certified time is the deployment-relevant quantity: the builder needs the system to succeed with probability $1 - \delta$ within a time budget, not merely to succeed "on average." The log of certified time, $\log \overline{T}_A$, is the natural scale for additive decomposition.

**Definition 5 (Certified log-speedup)** *Given a baseline solver $A_0$ and a deployed solver $A$, the certified log-speedup is $\Delta = \log \overline{T}_{A_0}(n, \delta) - \log \overline{T}_A(n, \delta)$. Positive $\Delta$ means the deployed solver is faster.*

### 3.4 The packed latent-mode family

The exact decomposition requires a structural assumption on the task family. The assumption is stylized but captures the essential feature of verifiable tasks with multiple solution strategies.

**Assumption 6 (Packed latent-mode family)** *Fix a task family at scale $n$. Assume:*

1. ***Latent modes.*** *There is a discrete latent variable $S \in [M_n] := \{1, \ldots, M_n\}$ with prior $\pi$, representing the solution strategy, and an observable $D \sim P(D \mid S)$ representing verifier feedback and partial execution traces, with posterior $\pi_D(s) := \mathbb{P}[S = s \mid D]$.*
2. ***One expert per mode.*** *Each mode $s$ has a specialized expert with certified internal scale $t_0(s)$.*
3. ***Allocation determines time.*** *A routing policy chooses a share vector $q \in \Delta_{M_n}$. Under hidden mode $s$, allocating share $q(s)$ yields certified time*

$$\overline{T}_q(s) \;=\; c_{\text{sim}} \cdot \frac{t_0(s)}{q(s)},$$

*where $c_{\text{sim}}$ is the simulation overhead.*

4. ***Baseline and deployed schedulers.*** *The baseline chooses a fixed share vector $q_0 \in \Delta_{M_n}$; the deployed system chooses a data-dependent map $D \mapsto q_D \in \Delta_{M_n}$.*

In Lean tactic prediction, the modes are proof strategy families: induction, case splitting, library search, simp chains, rewrite sequences. The observable $D$ is the partial execution trace—which tactics were tried, which failed, what error messages the kernel returned. The allocation $q(s)$ is the fraction of the attempt budget devoted to strategy $s$. The assumption that $\overline{T}_q(s) \propto t_0(s)/q(s)$ is a geometric-trials model: doubling the share on the correct strategy halves expected time to find the proof. This is well-supported empirically for search-based theorem proving [17].

**The entropy-effective mode count.** For a posterior $\pi_D$ over modes, define $M_{\text{eff}}(D) := \exp(H(\pi_D))$. When $M_{\text{eff}}$ is small on most instances, only a few modes need to be separated and executed well—the regime where specialization is most valuable.

### 3.5 The graph is not the task

The following proposition formalizes why the task-graph separation matters for design decisions.

**Proposition 7 (Benchmark racing can be suboptimal)** *Under the packed-family assumption, consider two models $M_A$ and $M_B$ with distinct optimal routing policies: $\pi_D^{*,A} \neq \pi_D^{*,B}$ on a set of instances of positive measure. Then evaluating both models under a single fixed workflow (as benchmark racing does) can select the wrong model: the model with lower win rate under the fixed workflow may achieve lower certified time under its own optimal routing.*

**Proof** Model $M_A$ evaluated under its optimal routing has mismatch $\varepsilon_A(\pi^{*,A}) = 0$. Model $M_B$ evaluated under $M_A$'s routing has mismatch $\varepsilon_B(\pi^{*,A}) = \mathbb{E}_D[\mathrm{KL}(\pi_D^{*,B} \| \pi_D^{*,A})] > 0$. Benchmark racing compares both under $\pi^{*,A}$, penalizing $M_B$ by $\varepsilon_B(\pi^{*,A})$ nats. Under its own routing, $M_B$ recovers the full penalty. If $M_B$'s other advantages exceed $M_A$'s by less than $\varepsilon_B(\pi^{*,A})$ but more than zero, benchmark racing selects $A$ while $B$ with adapted routing is superior. ■

A concrete instance: Model A wins 60% of problems with a fixed workflow; Model B wins only 45% with the same workflow but 70% with an adapted routing policy. Benchmark racing selects A; the framework selects B. The graph is not the task. The task is the same in both cases; the graphs—and therefore the performance—differ because the routing policy differs.

**Scope declaration.** The framework restricts to verifiable tasks with external checkers. This already includes many of the most important agentic workloads: code synthesis against tests, formal theorem proving, constrained planning, and verifier-backed search. The restriction is a feature: it is precisely the regime in which time-to-solution becomes mathematically clean and engineering-relevant.

## 4 Agent Building as a Deployment Optimization Problem

The task object and solver-induced graph are now in place. The question is no longer only why a finished system is slow. It is how to choose model, memory, tools, routing, and parallelism before deployment.

### 4.1 The design tuple

A *design point* is

$$d = (M, \pi, \Omega, W, K, p) \in \mathfrak{D}_n, \tag{1}$$

where $M$ is the base model, $\pi$ is the orchestration/routing policy, $\Omega$ is the available tool library, $W$ denotes long-term memory resources, $K$ is the active context budget, and $p$ is the allowed degree of parallelism. Write $A_d$ for the solver induced by design $d$.

In the Lean example: $M$ is DeepSeek-Prover-7B or Claude Opus; $\pi$ is the tactic-selection and backtracking policy; $\Omega$ includes the Lean kernel, Mathlib search, and rewriting tools; $W$ is the persistent lemma cache; $K$ is the proof-state context window; and $p$ is the number of parallel proof branches.

### 4.2 The master optimization problem

For a confidence level $\delta \in (0, 1)$ and a deployment loss $\Lambda_n$ over feasible time–cost–memory–parallelism tuples, the builder solves

$$\boxed{\min_{d \in \mathfrak{D}_n} \Lambda_n(t, c, m, p) \qquad \text{subject to} \qquad (t, c, m, p) \in \mathcal{F}^{\text{dep}}_{A_d}(n, \delta)} \tag{2}$$

where $\mathcal{F}^{\text{dep}}_{A_d}(n, \delta)$ is the deployment frontier: the set of time–cost–memory–parallelism tuples at which solver $A_d$ succeeds with probability $\geq 1 - \delta$.

This is the sense in which building an agent system is an operations-research problem. The builder chooses a design point under resource constraints, not merely a single model under a scalar accuracy metric. The paper does not claim to solve this optimization in full generality. It identifies the design-variable space, provides a decomposable objective for the model-choice component, and solves restricted sub-problems (Sections 6–7).

### 4.3 Suboptimality of model-restricted optimization

Benchmark racing restricts the design program to model variation alone: it fixes the routing policy and all other coordinates, then compares models. The following proposition quantifies what this restriction forfeits.

**Proposition 8 (Suboptimality of $M$-restricted optimization)** *Under the packed-family assumption (Assumption 6), consider two candidate models $M_A$ and $M_B$ with distinct optimal routing policies: $\pi^{*,A}_D \neq \pi^{*,B}_D$ on a set of instances of positive measure. If the builder restricts to $M$-variation alone—fixing $\pi = \pi^{*,A}$ and all other coordinates at model $A$'s values—then the suboptimality gap relative to the unrestricted optimum is at least*

$$\varepsilon_B(\pi^{*,A}) - \varepsilon_B(\pi^{*,B}) \;=\; \varepsilon_B(\pi^{*,A}) \;>\; 0,$$

*the routing-mismatch penalty from evaluating model $B$ under the wrong routing policy.*

**Proof** Under the four-term decomposition (Theorem 21), the certified log-speedup for model $M_B$ under routing policy $q$ is $\Delta(M_B, q) = \log(\kappa_0/\kappa_B) + \varphi_B + G_B - \varepsilon_B(q)$, where $\varepsilon_B(q) = \mathbb{E}_D[\mathrm{KL}(\pi_D^{*,B} \| q_D)]$. The unrestricted optimum uses $q = \pi_D^{*,B}$, achieving $\varepsilon_B(\pi^{*,B}) = 0$. Under $M$-restriction, the builder evaluates $M_B$ with $\pi^{*,A}$, incurring $\varepsilon_B(\pi^{*,A}) = \mathbb{E}_D[\mathrm{KL}(\pi_D^{*,B} \| \pi_D^{*,A})] > 0$ since the policies differ on positive measure and KL divergence is zero if and only if the distributions agree almost surely. ■

**Corollary 9 (Benchmark racing eliminates routing-adaptive designs)** *Benchmark racing can only discover which model is better under one fixed routing policy. It cannot discover that model $B$ with an adapted routing policy outperforms model $A$—precisely the design the framework identifies as optimal in the contested zone where $M_{\text{eff}} \approx M_{\text{eff}}^{\star}$.*

**Builder interpretation.** The suboptimality gap is largest when two models' optimal routing policies differ substantially—the contested zone where the framework's marginal value over benchmark racing is concentrated. In the clear-cut zones ($M_{\text{eff}} \ll M_{\text{eff}}^{\star}$ or $M_{\text{eff}} \gg M_{\text{eff}}^{\star}$), one model dominates regardless of routing, and benchmark racing gives the right answer.

### 4.4 What does posing the problem this way buy?

Without the OR framing, the builder does trial-and-error over a combinatorial design space. With it, the builder has structured search with named dimensions: model choice affects cost and competence; routing policy affects mismatch; tools and memory affect available structure; context budget trades node cost against graph shrinkage; parallelism trades coordination overhead against critical-path reduction. The four-term decomposition provides the objective function for the model-choice component.

**Which sections address which coordinates.** Model choice $M$ is addressed by Section 7 (crossover theorem under simplifying assumptions). Routing policy $\pi$ is partially addressed by Section 5 (orchestration equivalence: when two tasks demand the same routing) and Section 6 (the $\varepsilon$ term diagnoses routing mismatch). Transfer across task distributions is characterized by Section 5. Orchestration architecture $\Omega$, context budget $W$, and parallelism $p$ are addressed only partially. Full joint optimization over all six coordinates simultaneously, multi-agent team composition beyond the threshold result, and optimal budget allocation under cost constraints remain open problems.

**Closing promise.** The design program identifies six decision variables. The next two sections show that three of these variables are often redundant—because tasks that appear different are, for design purposes, the same—and that the remaining objective decomposes exactly into four terms a builder can estimate and independently improve.

## 5 When Are Two Tasks the Same?

Benchmark racing treats every new task as an independent evaluation problem. The equivalence framework reveals when two apparently different tasks are the same design problem—and when training on one transfers to the other.

### 5.1 Budget-preserving reduction and task equivalence

**Definition 10 (Budget-preserving reduction)** *A* budget-preserving reduction *from task family $\mathfrak{T}^A$ to task family $\mathfrak{T}^B$ at scale $n$ is a pair of maps $(f, g)$ where $f : \mathcal{X}_n^A \to \mathcal{X}_n^B$ maps instances and $g : \mathcal{Y}_n^B \to \mathcal{Y}_n^A$ maps solutions, such that:*

1. ***Correctness preservation:*** $V_n^B(f(x), y) = 1 \implies V_n^A(x, g(y)) = 1$.
2. ***Budget preservation:*** *the composition $x \mapsto g(A_d^B(f(x)))$ respects the budget constraints of $\mathfrak{T}^A$: the overhead of applying $f$ and $g$ is absorbed within $\mathcal{B}_n^A$.*

**Definition 11 (Strong and weak equivalence)** *Two task families $\mathfrak{T}^A$ and $\mathfrak{T}^B$ are:*

- Strongly equivalent *if budget-preserving reductions exist in both directions with overhead that is $o(1)$ relative to the budgets.*
- $\alpha$-weakly equivalent *if for every design $d$ the certified log-speedups satisfy $|\Delta^A(d) - \Delta^B(d)| \leq \alpha$.*

Strong equivalence says the two tasks are the same design problem: any solver for one is a solver for the other at negligible overhead. Weak equivalence is looser: the two tasks may differ, but no design choice exploits the difference by more than $\alpha$ nats.

### 5.2 Post-training equivalence

The builder's question: if a model is post-trained on task family $A$ and deployed on family $B$, should the builder expect the same performance?

**Definition 12 (Post-training efficiency)** *Fix a base solver $U^{(0)}$ and an update rule* Upd. *For data budget $m$, the* post-training efficiency of $A$-data for task $B$ *is the transfer ratio*

$$\eta_{A\to B}(m) \;:=\; \frac{\Delta_{A\to B}(n, m, \delta)}{\Delta_{B\to B}(n, m, \delta)}.$$

*When $\eta_{A\to B} \approx 1$, training on $A$-data is almost as good as training on $B$-data for improving performance on $B$.*

**Theorem 13 (Post-training equivalence from task equivalence)** *In the packed-family regime, suppose both families share the same latent space $S \in [M_n]$ with different observation channels $P_A(D|S)$ and $P_B(D|S)$. Then*

$$\eta_{A\to B}(m) \;=\; \frac{I(S; D_A^{(m)}) - \varepsilon_{A\to B}}{I(S; D_B^{(m)}) - \varepsilon_{B\to B}}. \tag{3}$$

*In particular:*

(i) *If both families share all latent structure and cross-family mismatch equals within-family mismatch, then $\eta_{A\to B} = 1$: training on $A$ is exactly as good as training on $B$.*

(ii) *If the families share the latent space but different channels, then $\eta_{A\to B} \approx I(S; D_A)/I(S; D_B)$ when mismatch terms are small: the transfer ratio is governed by the information-quality ratio of the two channels.*

**Proof** Apply the Routing-Information Identity (Theorem 19) to both transfer settings over the shared mode space. For the numerator, the side-information variable is $D_A^{(m)}$ drawn from the $A$-channel, applied to target family $B$:

$$\Delta_{A\to B} \;=\; I(S; D_A^{(m)}) - \varepsilon_{A\to B}, \quad \text{where } \varepsilon_{A\to B} := \mathbb{E}_{D_A}\big[\mathrm{KL}\big(\pi^B_{D_A} \| q_B(\cdot \mid D_A)\big)\big]\,.$$

Here $\pi^B_{D_A}(s) = \mathbb{P}[S = s \mid D_A]$ is the posterior over $B$-task modes given $A$-channel data. For the denominator, within-family data gives $\Delta_{B\to B} = I(S; D_B^{(m)}) - \varepsilon_{B\to B}$. Dividing yields (3). Claims (i) and (ii) follow by specialization. ■

**The shared-latent-space assumption is strong.** It is appropriate when both families draw on the same underlying skill bank—coding and formal mathematics both involve decomposition, symbolic manipulation, and verification. It is inappropriate when the families share only surface-level features.

**Design heuristic for mixed-source post-training.** For fraction $\alpha$ of data from source $A$, the natural objective is $\Delta_{\alpha\to B}(m) = I(S; D_\alpha^{(m)}) - \varepsilon_\alpha$. The mutual information is concave in $\alpha$ by standard arguments [11]. The behavior of $\varepsilon_\alpha$ is less clear: an earlier draft asserted convexity by citing "the infimum of convex functions is convex," which is false in general. We state as a design heuristic: when the concavity of $I(S; D_\alpha)$ dominates any non-convexity in $\varepsilon_\alpha$, compute $\Delta_{\alpha\to B}$ on a coarse grid and select the maximizer. Whether the objective is generically concave under natural structural conditions is an open question (Section 10).

### 5.3 Transfer characterization

**Proposition 14 (Transfer through target-relevant side information)** *Suppose the target family $\mathfrak{T}^B$ admits the packed-family representation with latent mode $S_B \sim \pi^B$. Let adaptation on family $\mathfrak{T}^A$ produce a random representation $Z_A$, and let the transferred system choose allocation $q_B(\cdot \mid Z_A)$. Write $\pi^B_{Z_A}(s) := \mathbb{P}[S_B = s \mid Z_A]$. Then the packed-family certified-time gain on $\mathfrak{T}^B$ from using $Z_A$ is*

$$\Delta^{\text{packed}}_{A\to B} \;=\; \mathrm{I}_{\pi^B}(S_B : Z_A) \;-\; \mathbb{E}\big[\mathrm{KL}\big(\pi^B_{Z_A} \| q_B(\cdot \mid Z_A)\big)\big]\,. \tag{4}$$

**Proof** Apply the packed-family decomposition to the target family $\mathfrak{T}^B$ with side-information variable $Z_A$ rather than within-family data. The proof is otherwise unchanged: the cross-entropy identity splits the allocation term into information and mismatch. ■

**Remark 15 (DPI bound under joint latent structure)** *If additionally $D_A \leftarrow S_A \leftrightarrow S_B \to D_B$ is a Markov chain and $Z_A$ is a function of $(D_A, S_A)$, then by the data-processing inequality: $\Delta^{\text{packed}}_{A\to B} \le \mathrm{I}(S_A : S_B)$. Transfer is bounded by the mutual information between the latent mode spaces. The Markov chain assumption is appropriate when the families share a common underlying skill space; it is inappropriate when they share only surface features.*

**Estimability.** The quantity $\mathrm{I}(S_B; Z_A)$ is a theoretical characterization, not an actionable prescription: estimation requires model internals or representation probing. For practical guidance, the builder uses the three-case conditional heuristic below and an estimable mode-overlap proxy—the Bhattacharyya coefficient between empirical mode distributions from two pilot runs. Estimation of $\mathrm{I}(S_B; Z_A)$ is an open problem (Section 10).

**Conditional guidance.** The framework yields three conditional prescriptions for transfer:

- *Shared dominant modes, similar orchestration.* Training benefit transfers. Deploy the source-trained model with minor adaptation.
- *Shared dominant modes, different orchestration.* Retrain the routing policy but keep the base model. Competence transfers; the routing does not.
- *Target has modes not present in the source.* Broader pre-training matters more than source-task fine-tuning. The source representation carries little information about the target's latent modes.

### 5.4 Orchestration equivalence

Post-training equivalence governs *what model to train.* Orchestration equivalence governs a complementary question: *when to reuse the same workflow.*

**Definition 16 ($\alpha$-Orchestration equivalence)** *Two packed-family task families $\mathcal{T}^A$ and $\mathcal{T}^B$ sharing a common mode bank $[M_n]$ are* $\alpha$-orchestration-equivalent *if*

$$\mathbb{E}_D\big[\mathrm{KL}\big(\pi_D^A \,\big\|\, \pi_D^B\big)\big] \;\le\; \alpha, \tag{5}$$

*where $\pi_D^A, \pi_D^B \in \Delta_{M_n-1}$ are the posterior mode distributions induced by side information $D$ under the two families, and the expectation is over the marginal distribution of $D$.*

Two families that are $\alpha$-orchestration-equivalent with small $\alpha$ demand similar routing policies: the posterior beliefs about which strategy to deploy are close on average.

**Proposition 17 (Orchestration equivalence bounds)** *Suppose $\mathcal{T}^A$ and $\mathcal{T}^B$ are $\alpha$-orchestration-equivalent packed families with common mode bank $[M_n]$. Then:*

***(a) Information-term bound*** *(independent of routing policy $q$).*

$$|G_A - G_B| \;=\; \big|\mathbb{E}_D[H(\pi_D^A)] - \mathbb{E}_D[H(\pi_D^B)]\big| \;\le\; \sqrt{2\alpha}\,\log\!\left(\frac{M_n}{\sqrt{2\alpha}}\right), \tag{6}$$

*provided $\sqrt{2\alpha} \le 1 - 1/M_n$.*

***(b) Routing-mismatch bound*** *(depends on routing policy $q$). For any routing policy $q$ with $q_{\min} := \min_s q(s) > 0$:*

$$|\varepsilon_A(q) - \varepsilon_B(q)| \;\le\; \sqrt{2\alpha}\,\log(1/q_{\min}) \;+\; \sqrt{2\alpha}\,\log\!\left(\frac{M_n}{\sqrt{2\alpha}}\right). \tag{7}$$

***(c) Simplified bound under uniform-floor routing.*** *If $q_{\min} \ge 1/M_n$, both bounds are $O(\sqrt{\alpha}\,\log M_n)$.*

*Proof sketch.* Apply Pinsker's inequality [30, 12] pointwise: $\|\pi_D^A - \pi_D^B\|_1 \leq \sqrt{2\,\mathrm{KL}(\pi_D^A \| \pi_D^B)}$. Take expectations and apply Jensen (concavity of $\sqrt{\cdot}$): $\bar{\delta} := \mathbb{E}[\|\pi_D^A - \pi_D^B\|_1] \leq \sqrt{2\alpha}$. For (a), apply Audenaert's [3] entropy continuity lemma: $|H(p) - H(r)| \leq \|p - r\|_1 \log(M_n/\|p - r\|_1)$ for distributions on $M_n$ atoms. Take expectations; $t \mapsto t \log(M_n/t)$ is concave, so Jensen gives the bound in terms of $\bar{\delta}$. For (b), use the KL-difference decomposition:

$$\mathrm{KL}(p\|q) - \mathrm{KL}(r\|q) = [H(r) - H(p)] + \textstyle\sum_s [p(s) - r(s)] \log(1/q(s)).$$

Bound the first term by (a) and the second by $\bar{\delta} \cdot \log(1/q_{\min})$ via Pinsker. Full proof in Appendix B.

**Builder interpretation.** If two tasks are $\alpha$-orchestration-equivalent, the penalty from reusing task $A$'s routing policy on task $B$ is at most $O(\sqrt{\alpha} \log M_n)$ nats in both the information and routing terms. For a system with $M_n = 10$ modes and $\alpha = 0.01$, this is about $2\sqrt{0.02} \cdot \log(10) \approx 0.65$ nats—less than the typical cost-ratio term $\log(30) \approx 3.4$ nats. The bound degrades with $\sqrt{\alpha}$, not $\alpha$, reflecting the sub-linear relationship between KL divergence and total-variation distance. This is weaker than a linear bound for small $\alpha$ but is the correct scaling from first principles.

### 5.5 The two equivalences are independent

The two equivalence notions capture independent design dimensions.

**Orchestration-equivalent but not post-training-equivalent.** Two families can share the same posterior structure (same beliefs about which mode to use, hence small $\alpha$) while one requires much more data to reach those beliefs. The posteriors are close; the data requirements are not. A builder can reuse the workflow but must invest in separate training.

**Post-training-equivalent but not orchestration-equivalent.** Two families can share the same latent modes (training on one transfers to the other, so $\eta_{A \to B} \approx 1$) while the posterior structures differ enough that they demand different routing policies. Coding and formal mathematics may share symbolic-manipulation modes, so fine-tuning transfers. But the optimal proof-search orchestration differs from the optimal code-repair orchestration. A builder can reuse the model but must redesign the workflow.

This separation is not a pathology. It is the conceptual payoff of having two distinct equivalence notions: post-training equivalence tells the builder when to reuse training data; orchestration equivalence tells the builder when to reuse the workflow. The decomposition in the next section provides the objective these equivalences operate on.

## 6 Information-Routing and Model-Aware Decompositions

The equivalence notions of the previous section identify when two tasks are the same design problem. The present section provides the objective function that design problem optimizes. We first establish the Routing-Information Identity—the two-term decomposition $\Delta = G - \varepsilon$ that is the paper's cleanest equation—then extend it to the four-term Model-Aware Decomposition that compares models, and finally prove the upper envelope theorem and the approximation guarantee outside the packed family.

### 6.1 The Routing-Information Identity

The first result decomposes certified log-speedup into two terms: how much the system learns about the latent mode, and how well it uses what it learns.

**Lemma 18 (Conditional log-time decomposition)** *Under Assumption 6, for any realized feedback $D$ and any share vector $q \in \Delta_{M_n}$ with* $\mathrm{supp}(\pi_D) \subseteq \mathrm{supp}(q)$,

$$\mathbb{E}_{S\sim\pi_D}\big[\log \overline{T}_q(S) \mid D\big] \;=\; \mathbb{E}_{S\sim\pi_D}[\log(c_{\mathrm{sim}}\, t_0(S))] + H(\pi_D) + \mathrm{KL}(\pi_D\|q).$$

*The allocation $q = \pi_D$ uniquely minimizes conditional expected certified log-time. The excess over this optimum is exactly* $\mathrm{KL}(\pi_D\|q)$.

**Proof** Since $\log \overline{T}_q(s) = \log(c_{\mathrm{sim}}\, t_0(s)) - \log q(s)$,

$$\mathbb{E}_{S\sim\pi_D}\big[\log \overline{T}_q(S) \mid D\big] = \mathbb{E}_{S\sim\pi_D}[\log(c_{\mathrm{sim}}\, t_0(S))] + \sum_s \pi_D(s) \log \frac{1}{q(s)} = \mathbb{E}_{S\sim\pi_D}[\log(c_{\mathrm{sim}}\, t_0(S))] + H(\pi_D) + \mathrm{KL}(\pi_D\|$$

using the cross-entropy identity $\sum_s p(s)\log(1/q(s)) = H(p) + \mathrm{KL}(p\|q)$. Nonnegativity of KL divergence gives the optimality claim. ■

**Theorem 19 (Routing-Information Identity)** *Under Assumption 6, with a prior-matched baseline $q_0 = \pi$,*

$$\boxed{\Delta \;=\; G - \varepsilon}$$

*where*

$$G := \mathrm{I}_\pi(S : D) = H(\pi) - \mathbb{E}_D[H(\pi_D)], \tag{8}$$

$$\varepsilon := \mathbb{E}_D[\mathrm{KL}(\pi_D\|q_D)]\,. \tag{9}$$

*The term $G$ is the mutual information between the latent mode and the feedback—the* information generation quality*. The term $\varepsilon$ is the expected KL divergence of the routing policy from the posterior—the* routing mismatch*. Equivalently: $\Delta \le G$, with equality if and only if $q_D = \pi_D$ almost surely. One nat of routing mismatch costs exactly one nat of certified log-speedup.*

**Proof** Apply Lemma 18 to the baseline allocation $q_0 = \pi$:

$$\mathbb{E}\big[\log \overline{T}_\pi(S)\big] = \mathbb{E}_\pi[\log(c_{\mathrm{sim}}\, t_0(S))] + H(\pi).$$

Apply it to the deployed allocation $q_D$:

$$\mathbb{E}\big[\log \overline{T}_{q_D}(S)\big] = \mathbb{E}_\pi[\log(c_{\mathrm{sim}}\, t_0(S))] + \mathbb{E}_D[H(\pi_D)] + \mathbb{E}_D[\mathrm{KL}(\pi_D\|q_D)].$$

Subtract:

$$\Delta = H(\pi) - \mathbb{E}_D[H(\pi_D)] - \mathbb{E}_D[\mathrm{KL}(\pi_D\|q_D)] = G - \varepsilon.$$

■

The identity is algebraic: no approximation, no limit, no asymptotic qualifier. In the packed-family regime, information generation and routing mismatch are the *only* determinants of speedup, and they combine additively on the log scale. A system can be fast because it generates good information ($G$ large) or slow because it routes poorly ($\varepsilon$ large), and these two failures are separable.

For a non-prior-matched baseline $q_0 \neq \pi$, the identity becomes $\Delta = G + \mathrm{KL}(\pi \| q_0) - \varepsilon$, adding a baseline suboptimality bonus. The prior-matched case is the clean comparison.

### 6.2 The Model-Aware Decomposition

The Routing-Information Identity holds model fixed. To compare models, we extend the packed-family assumption to include model-dependent costs and competence.

**Assumption 20 (Model-dependent packed family)** *Fix a task family and a shared latent mode variable $S \sim \pi$ over $[M_n]$. For each model $M$:*

1. *The per-step cost is $\kappa(M)$.*
2. *The within-mode certified scale is $t_0(M, s)$, so that certified time under mode $s$ with allocation share $q(s)$ is*
$$\overline{T}_q^{(M)}(s) = \kappa(M) \cdot c_{\mathrm{sim}} \cdot \frac{t_0(M, s)}{q(s)}.$$
3. *The feedback observable is $D \sim P_M(D \mid S)$, with posterior $\pi_D^M(s) := \mathbb{P}[S = s \mid D_M = D]$.*
4. *The routing policy is $q_D^M \in \Delta_{M_n}$.*

**Theorem 21 (Model-Aware Decomposition)** *Under Assumption 20, comparing a prior-matched baseline using model $M_0$ against a deployed system using model $M$,*

$$\boxed{\Delta_M \;=\; \underbrace{\log \frac{\kappa(M_0)}{\kappa(M)}}_{\textit{per-step cost}} + \underbrace{\varphi(M_0, M)}_{\textit{competence}} + \underbrace{G(M)}_{\textit{information}} - \underbrace{\varepsilon(M)}_{\textit{routing mismatch}}} \tag{10}$$

*where the four terms are:*

$$\log \frac{\kappa(M_0)}{\kappa(M)} \quad \textit{(per-step cost advantage of } M\textit{)}, \tag{11}$$

$$\varphi(M_0, M) := \mathbb{E}_\pi\left[\log \frac{t_0(M_0, S)}{t_0(M, S)}\right] \quad \textit{(within-mode competence advantage)}, \tag{12}$$

$$G(M) := \mathrm{I}_\pi(S : D_M) \quad \textit{(information generation quality)}, \tag{13}$$

$$\varepsilon(M) := \mathbb{E}_D\big[\mathrm{KL}(\pi_D^M \| q_D^M)\big] \quad \textit{(routing mismatch)}. \tag{14}$$

**Proof** Write out the log-time:

$$\log \overline{T}_q^{(M)}(s) = \log \kappa(M) + \log c_{\mathrm{sim}} + \log t_0(M, s) - \log q(s).$$

For the prior-matched baseline with model $M_0$:

$$\mathbb{E}\Big[\log \overline{T}_{\pi}^{(M_0)}(S)\Big] = \log \kappa(M_0) + \log c_{\text{sim}} + \mathbb{E}_{\pi}[\log t_0(M_0, S)] + H(\pi).$$

For the deployed system with model $M$:

$$\begin{aligned}\mathbb{E}\Big[\log \overline{T}_{q_D}^{(M)}(S)\Big] &= \log \kappa(M) + \log c_{\text{sim}} + \mathbb{E}_{\pi}[\log t_0(M, S)] \\ &\quad + \mathbb{E}_D[H(\pi_D^M)] + \mathbb{E}_D[\mathrm{KL}(\pi_D^M \| q_D^M)],\end{aligned}$$

where the last two terms arise from the cross-entropy identity exactly as in Theorem 19. Subtracting and grouping:

$$\begin{aligned}\Delta_M &= \log \frac{\kappa(M_0)}{\kappa(M)} + \mathbb{E}_{\pi}\left[\log \frac{t_0(M_0, S)}{t_0(M, S)}\right] + \big(H(\pi) - \mathbb{E}_D[H(\pi_D^M)]\big) - \mathbb{E}_D[\mathrm{KL}(\pi_D^M \| q_D^M)] \\ &= \log \frac{\kappa(M_0)}{\kappa(M)} + \varphi(M_0, M) + G(M) - \varepsilon(M).\end{aligned}$$

■

The four terms have clean interpretations:

- **Per-step cost** $\log(\kappa_0/\kappa)$: the log cost ratio. A 7B model at 1/30th the price contributes $\log 30 \approx 3.4$ nats.
- **Competence** $\varphi$: the expected log-ratio of within-mode solve times. A fine-tuned specialist that is twice as fast per mode contributes $\log 2 \approx 0.7$ nats.
- **Information** $G$: the mutual information between the latent mode and verifier feedback. A task with 8 equiprobable modes and a verifier that identifies the correct one contributes $\log 8 \approx 2.1$ nats.
- **Routing mismatch** $\varepsilon$: the KL divergence of the routing policy from the posterior. A policy that spreads mass uniformly when the posterior is concentrated wastes $\varepsilon$ nats of potential speedup.

Benchmark racing reports only $\Delta$. The decomposition reports *why* $\Delta$ takes the value it does. Two systems with the same $\Delta$ may differ entirely in how they achieve it—one through low cost, the other through good routing—and the appropriate intervention is different in each case.

### 6.3 The upper envelope theorem

The decomposition is exact within the packed family. Before specializing further, we state the general upper bound that holds outside it. The theorem rests on the following regularity conditions on the solver space and a structural witness.

**Assumption 22 (A1–A7: shared admissibility and aligned witness)**

- ***A1*** *(verifiability):* $V_n \in \mathrm{P}$.

- ***A2*** *(prefix coding): $\mathcal{A}_n$ is countable and prefix-free, and both $w(\cdot)$ and $w(\cdot \mid D_n)$ satisfy Kraft.*
- ***A3*** *(shared admissibility): the same nonempty set $\mathcal{A}_n^\delta$ is used in the baseline and conditioned schedule objectives.*
- ***A4*** *(shared simulation): both schedules incur the same simulation overhead $c_{\mathrm{sim}}(n) \in \mathrm{poly}(n)$.*
- ***A5*** *(resource-bounded conditioning): the map $D_n \mapsto w(\cdot \mid D_n)$ is computable or decodable in polynomial time.*
- ***A6*** *(structure witness class): there is a nonempty subclass $S(H_n) \subseteq \mathcal{A}_n^\delta$ of admissible solvers that realize the reusable structure $H_n$.*
- ***A7*** *(aligned witness): there exists $a^\star \in S(H_n)$ and nonnegative slacks $(\rho_U, \rho_D, \kappa_U)$ such that*

$$F_U(a^\star) \le e^{\rho_U} \inf_{a \in \mathcal{A}_n^\delta} F_U(a), \qquad F_D(a^\star) \le e^{\rho_D} \inf_{a \in \mathcal{A}_n^\delta} F_D(a),$$

$$-\log w(a^\star) \le K^{\mathrm{poly}}(H_n) + \kappa_U,$$

*where $F_U$ and $F_D$ denote the baseline and conditioned schedule objectives and $K^{\mathrm{poly}}(H_n) := \inf_{a \in S(H_n)}\{-\log w(a)\}$ is the resource-bounded Kolmogorov complexity of the reusable structure.*

**Assumption 23 (Sufficient-statistic condition)** *The history $H_n$ is a* sufficient statistic *for the strategy set $S(H_n)$ relative to the side information $D_n$:*

$$P(D_n \mid H_n, S(H_n)) = P(D_n \mid H_n).$$

*This assumption is natural in the packed-family setting, where $H_n$ determines the posterior allocation over modes and thereby determines the distribution of strategies.*

**Theorem 24 (Upper envelope at expectation level)** *Under Assumptions A1–A7 (Assumption 22) and the sufficient-statistic condition (Assumption 23):*

$$\mathbb{E}_{D_n}\big[\mathrm{Sp}(D_n)\big] \;\le\; I(H_n; D_n) + \log\big(1/w(S(H_n))\big),$$

*where $\mathrm{Sp}(D_n) := \log(T_U/T_{U_{D_n}})$ is the log-speedup from conditioning on $D_n$, $I(H_n; D_n)$ is the mutual information between the history and the side information, and $w(S(H_n))$ is the prior weight assigned to the strategy set.*

**Proof** *Step 1 (Pointwise ratio bound).* By Assumptions A1–A7, there exists $a^* \in S(H_n)$ such that for each realization of $D_n$:

$$\mathrm{Sp}(D_n) \;\le\; \rho_{D_n} + \log w(a^* \mid D_n) - \log w(a^*).$$

*Step 2 (Expectations under sufficiency).* Take expectations over $D_n$. The sufficient-statistic condition ensures that $a^*$ does not depend on $D_n$ in a way that correlates with the posterior weights beyond the dependence through $H_n$, so Jensen's inequality applies cleanly:

$$\mathbb{E}_{D_n}\big[\log w(a^* \mid D_n)\big] \;\le\; \mathbb{E}_{D_n}\big[\sup_{a \in S(H_n)} \log w(a \mid D_n)\big].$$

*Step 3 (Information bound).* By the Kraft inequality applied to both prior and posterior:

$$\mathbb{E}_{D_n}[\rho_{D_n}] + \mathbb{E}_{D_n}\big[\sup_{a\in S(H_n)} \log w(a \mid D_n)\big] - \log w(a^*) \;\le\; I(H_n; D_n) + \log(1/w(S(H_n))).$$

*Step 4 (Combine).* Collecting the bounds:

$$\mathbb{E}_{D_n}\big[\mathrm{Sp}(D_n)\big] \;\le\; I(H_n; D_n) + \log(1/w(S(H_n))).$$

■

**Which results inherit the sufficiency assumption.** The sufficient-statistic condition is required for this upper envelope and inherited by the approximation guarantee (Proposition 25), which references the upper envelope as a ceiling. The RI Identity and the four-term decomposition are algebraic identities within the packed family and do *not* depend on the sufficiency assumption.

**Route A2 fallback.** If the sufficiency assumption is relaxed, a maximal-inequality route gives a weaker bound with an additional $\log|S(H_n)|$ term arising from a union bound over the strategy set.

### 6.4 Approximation guarantee outside the packed family

The packed-family assumptions are stylized. Real tasks have soft mode posteriors and approximate log-linear time scaling. The following proposition quantifies the degradation.

**Proposition 25 (Approximation guarantee)** *Suppose the packed-family assumptions (Assumption 20) hold except for two relaxations:*

1. (Soft modes.) *The posterior $\pi_D^M$ is $\delta$-approximately one-hot: there exists $s^*(D) \in [M_n]$ with $\pi_D^M(s^*(D)) \ge 1-\delta$ for some $\delta \in [0,1)$.*
2. (Approximate log-linearity.) *The certified time satisfies $\log \overline{T}_q^{(M)}(s) = \log\kappa(M) + \log c_{\mathrm{sim}} + \log t_0(M,s) - \log q(s) + r(M,s,q)$ where $|r(M,s,q)| \le \eta$ uniformly.*

*Then the four-term identity (Theorem 21) holds up to a remainder:*

$$\left|\mathbb{E}\Big[\log \overline{T}_\pi^{(M_0)}(S) - \log \overline{T}_{q_D}^{(M)}(S)\Big] - \left[\log\frac{\kappa(M_0)}{\kappa(M)} + \varphi(M_0,M) + G(M) - \varepsilon(M)\right]\right| \;\le\; R(\delta,\eta,M_n),$$

*where*

$$R(\delta,\eta,M_n) \;\le\; 2\delta\log M_n + 2\eta.$$

**Proof** The two relaxations contribute independently.

*Soft modes.* When $\pi_D^M$ is $\delta$-approximately one-hot, $H(\pi_D^M) \le h(\delta) + \delta\log(M_n-1) \le h(\delta) + \delta\log M_n$, where $h(\delta) = -\delta\log\delta - (1-\delta)\log(1-\delta)$. For $M_n \ge \exp(h(\delta)/\delta)$, we have $H(\pi_D^M) \le 2\delta\log M_n$. Both the baseline and deployed terms contribute, giving a routing correction of at most $2\delta\log M_n$.

*Approximate log-linearity.* The residual $r(M,s,q)$ enters additively into each log-time term. Since the identity subtracts two such terms, $|r(M,s,q_D) - r(M_0,s,\pi)| \le 2\eta$.

Combining: $R(\delta,\eta,M_n) \le 2\delta\log M_n + 2\eta$. ■

**When the bound is tight.** For $M_n = 8$ modes, $\delta = 0.1$, $\eta = 0.05$: $R \leq 0.42 + 0.10 = 0.52$ nats. Against a typical decomposition value $\Delta \approx 3$ nats (a 20x speedup), the remainder is about 17%—meaningful but not invalidating. The bound is tight when the effective mode count is small and the time model is approximately multiplicative.

## 7 Consequences for Model Choice and System Design

The decomposition is the accounting identity. Five consequences follow: a crossover formula, a specialization-dominance checklist, verifier equalization, a multi-agent threshold, and a Kelly-betting interpretation.

### 7.1 The crossover formula and effective mode count

**Corollary 26 (Model-selection crossover)** *Under Assumption 20, model $M_1$ achieves lower expected certified log-time than model $M_2$ whenever*

$$\underbrace{\log \frac{\kappa(M_2)}{\kappa(M_1)}}_{\textit{cost savings}} + \underbrace{\varphi(M_0, M_1) - \varphi(M_0, M_2)}_{\textit{competence advantage}} > \underbrace{G(M_2) - G(M_1)}_{\textit{information advantage}} + \underbrace{\varepsilon(M_1) - \varepsilon(M_2)}_{\textit{routing penalty}}. \tag{15}$$

**Proof** Apply Theorem 21 to both models with the same baseline; subtract. The baseline terms cancel. ■

**Derivation of the exponential threshold formula.** Equation (15) gives the general crossover condition. To obtain a closed-form threshold in terms of the effective mode count $M_{\text{eff}}$, we make three additional assumptions:

(S1) *Verifier equalization*: $|G(M_1) - G(M_2)| \leq \eta$ for $\eta$ small relative to $\log(\kappa_2/\kappa_1)$ (Proposition 27), so the information terms approximately cancel.

(S2) *Linear routing-mismatch scaling*: the specialist's routing mismatch grows approximately linearly with the log of the effective mode count, $\varepsilon(M_{\text{small}}) - \varepsilon(M_{\text{big}}) \approx \hat{c} \cdot \log M_{\text{eff}}$, where $\hat{c} > 0$ is the *per-mode routing penalty*. This holds exactly when the specialist allocates uniformly over $M_{\text{eff}}$ modes while the generalist routes according to the posterior, giving $\hat{c} = 1$; more generally, $\hat{c}$ is an empirically estimated coefficient.

(S3) *Net competence advantage $\hat{\varphi}$ independent of $M_{\text{eff}}$*: the within-mode competence difference $\varphi(M_0, M_1) - \varphi(M_0, M_2)$ is approximately constant across task strata.

Under (S1)–(S3), the crossover condition becomes $\log(\kappa_{\text{big}}/\kappa_{\text{small}}) + \hat{\varphi} > \hat{c} \cdot \log M_{\text{eff}}$, which gives the threshold:

$$M^{\star}_{\text{eff}} = \exp\left(\frac{\log(\kappa_{\text{big}}/\kappa_{\text{small}}) + \hat{\varphi}}{\hat{c}}\right). \tag{16}$$

For a cost ratio of 30x, comparable competence ($\hat{\varphi} \approx 0$), and binary verifier feedback ($\hat{c} = \log 2$, corresponding to one bit of routing penalty per mode doubling):

$$M^{\star}_{\text{eff}} = \exp\left(\frac{\log 30}{\log 2}\right) = 2^{\log_2 30} = 30.$$

Tasks with fewer than 30 effective modes belong to the specialist. With richer verifier feedback ($\hat{c} > \log 2$), the threshold drops: a more informative verifier makes it harder for the specialist to justify its cost advantage.

### 7.2 Specialization dominance

The crossover formula gives a threshold. The following checklist gives *sufficient conditions* for specialization dominance that are individually checkable.

A smaller model dominates when:

(i) it is cheaper: $\kappa_{\text{small}} < \kappa_{\text{big}}$;

(ii) it is comparably competent on effective modes: $\varphi(M_0, M_{\text{small}}) \geq \varphi(M_0, M_{\text{big}})$;

(iii) its routing mismatch is no worse: $\varepsilon_{\text{small}} \leq \varepsilon_{\text{big}}$;

(iv) verifier equalization holds: $|G_{\text{big}} - G_{\text{small}}|$ is small relative to the cost ratio.

When all four conditions hold, the specialist dominates on every channel simultaneously. The checklist is a design diagnostic, not a discovery: the conditions individually assert the small model is better on every dimension. The value lies in the *decomposition into four named, measurable channels* and the *explicit quantitative form* of the crossover.

### 7.3 Verifier equalization

The information term $G(M) = \mathrm{I}_\pi(S : D_M)$ is, in general, model-dependent: different models generate different probes and receive different verifier feedback. A key simplification occurs when the verifier is rich enough to make $G$ approximately model-independent.

**Proposition 27 (Verifier equalization)** *Under Assumption 20, suppose both models' residual posterior entropies are small:*

$$\mathbb{E}_D[H(\pi_D^{M_1})] \leq \eta_1, \qquad \mathbb{E}_D[H(\pi_D^{M_2})] \leq \eta_2.$$

*Then*

$$|G(M_1) - G(M_2)| \leq \eta_1 + \eta_2.$$

**Proof** $G(M) = H(\pi) - \mathbb{E}_D[H(\pi_D^M)]$. Since $H(\pi)$ is model-independent, $|G(M_1) - G(M_2)| = |\mathbb{E}_D[H(\pi_D^{M_2})] - \mathbb{E}_D[H(\pi_D^{M_1})]| \leq \eta_1 + \eta_2$. ■

Verifier equalization holds when the external checker provides enough feedback that both models' residual uncertainties about the latent mode are small. Formal proof assistants with step-level error messages, comprehensive test suites with diagnostic output, and constraint solvers with counterexample feedback are all settings where this is plausible. When verifier equalization holds, the crossover formula simplifies: the information terms nearly cancel, and the model-selection decision reduces to cost plus competence versus routing quality. The verifier is the great equalizer: a rich external checker neutralizes the large model's informational advantage, leaving only per-step cost and competence to determine the winner.

### 7.4 Multi-agent specialization threshold

The crossover formula compares two models. The multi-agent threshold extends the analysis to teams of specialized agents versus a single generalist.

**Assumption 28 (Multi-agent packed family with coordination cost)** *Consider a packed latent-mode family with $M_n$ modes and prior $\pi$. A single-agent design uses model $M_g$ (generalist) with per-mode competence $t_0(M_g, s)$, per-step cost $\kappa(M_g)$, and routing allocation $q_g$. A $K$-agent specialist-team design assigns each of $K$ agents to a subset of modes. Agent $i$ has model $M_i$ with per-mode competence $t_0(M_i, s)$ and per-step cost $\kappa(M_i)$. The team allocation is $q_{\text{team}}$, and $\text{team}(s) \in [K]$ denotes the agent assigned to mode $s$. The team incurs a coordination cost $c_{\text{coord}}(K) \geq 0$ added to the expected log-time. The prior-weighted average per-step cost of the team is $\bar{\kappa}_{\text{team}} := \sum_{s=1}^{M_n} \pi(s)\, \kappa(M_{\text{team}(s)})$.*

**Theorem 29 (Multi-agent specialization threshold)** *Under Assumption 28, suppose $G(M_g) - G_{\text{team}} = \gamma$ for a known constant $\gamma$. Then the $K$-agent specialist team achieves strictly lower expected certified log-time than the single generalist if and only if*

$$\underbrace{\sum_s \pi(s) \log \frac{t_0(M_g, s)}{t_0(M_{\text{team}(s)}, s)}}_{\phi_{\text{spec}}\text{: competence gain}} + \underbrace{\log \frac{\kappa(M_g)}{\bar{\kappa}_{\text{team}}}}_{\text{per-step savings}} + \underbrace{(\varepsilon_g - \varepsilon_{\text{team}})}_{\text{routing improvement}} > c_{\text{coord}}(K) + \gamma.$$

**Proof** Apply Theorem 21 to both designs with the same prior-matched baseline. For the generalist: $\Delta_g = \log(\kappa_0/\kappa(M_g)) + \varphi(M_0, M_g) + G(M_g) - \varepsilon_g$. For the team, the effective per-step cost is $\bar{\kappa}_{\text{team}}$ and the effective competence is $\varphi(M_0, M_{\text{team}}) = \sum_s \pi(s) \log(t_0(M_0, s)/t_0(M_{\text{team}(s)}, s))$. The team incurs additive coordination cost: $\Delta_{\text{team}} = \log(\kappa_0/\bar{\kappa}_{\text{team}}) + \varphi(M_0, M_{\text{team}}) + G_{\text{team}} - \varepsilon_{\text{team}} - c_{\text{coord}}(K)$. Subtracting:

$$\Delta_{\text{team}} - \Delta_g = \log \frac{\kappa(M_g)}{\bar{\kappa}_{\text{team}}} + \phi_{\text{spec}} - \gamma + (\varepsilon_g - \varepsilon_{\text{team}}) - c_{\text{coord}}(K),$$

where $\phi_{\text{spec}} = \sum_s \pi(s) \log(t_0(M_g, s)/t_0(M_{\text{team}(s)}, s))$ and the $M_0$ terms cancel. The biconditional follows because the four-term decomposition is an algebraic identity. ■

**Corollary 30 (Multi-agent threshold under verifier equalization)** *If verifier equalization holds, $|G(M_g) - G_{\text{team}}| \leq \eta$, then the specialist team dominates whenever*

$$\phi_{\text{spec}} + \log \frac{\kappa(M_g)}{\bar{\kappa}_{\text{team}}} + (\varepsilon_g - \varepsilon_{\text{team}}) > c_{\text{coord}}(K) + \eta.$$

*This is a* sufficient *condition: the team may dominate even when the left-hand side falls below $c_{\text{coord}}(K) + \eta$, provided the actual information-gain difference $\gamma$ is less than $\eta$.*

**Proof** From Theorem 29, the team dominates iff the LHS exceeds $c_{\text{coord}}(K) + \gamma$. Since $\gamma \leq \eta$ by verifier equalization, the condition LHS $> c_{\text{coord}}(K) + \eta \geq c_{\text{coord}}(K) + \gamma$ is sufficient. ■

**Builder interpretation.** The exact biconditional requires knowing $\gamma$ exactly. In practice, verifier equalization provides only $|\gamma| \leq \eta$, and the corollary gives a conservative test: if the combined savings from specialization exceed $c_{\text{coord}}(K) + \eta$, the team is guaranteed to dominate. The gap between the conservative threshold and the exact threshold is at most $2\eta$. This split follows Radner's [31] tradition of distinguishing exact equilibrium characterizations from sufficient conditions for coordination gains.

### 7.5 The Kelly connection

The inner routing problem of the packed family—choosing the share allocation $q$ to minimize expected certified log-time given feedback $D$—is the same log-optimal portfolio problem that appears in Kelly betting [23], universal portfolio theory [11], and algorithm-portfolio scheduling [17]. The optimal allocation $q^*(\cdot \mid D) = \pi_D(\cdot)$ maximizes expected log-speedup just as the Kelly bet maximizes expected log-wealth growth. The routing mismatch $\varepsilon = \mathbb{E}_D[\mathrm{KL}(\pi_D \| q_D)]$ is the expected log-wealth loss from suboptimal betting. The mathematical identity is known; the contribution is its instantiation in the LLM agent setting, where it prescribes posterior-matched routing as the compute-optimal strategy and decomposes the gap from optimality into four measurable terms.

## 8 Application: Building Math Research Agents

The preceding sections develop the framework in domain-general terms. We now instantiate it for building agents for formal theorem proving in Lean 4. This is not a toy example. Teams at several organizations are actively building such systems, facing precisely the design choices the framework addresses: which model, what orchestration, when to fine-tune, and how to diagnose failures.

The framework yields concrete, differentiated prescriptions that vary across three identifiable task regimes within the same domain. The *same framework* applied to different regimes within formal theorem proving produces opposing recommendations—a feature, not a bug, since it reflects genuine structural differences in what makes these tasks hard.

### 8.1 Three-regime taxonomy

Formal theorem proving in Lean 4 is not a single task. The strategy space, the role of library knowledge, and the nature of the creative bottleneck vary dramatically across problem types. We identify three regimes, distinguished by their effective mode count $M_{\text{eff}}$ and the resulting implications for the decomposition terms.

**Routine tactic application** ($M_{\text{eff}} \approx 3$–$5$). These are lemmas solvable by a small number of well-known tactic patterns: induction followed by simplification, ring arithmetic, omega for linear integer goals. The mode space is narrow, the verifier provides rich step-level feedback, and a specialist trained on Lean tactics has seen thousands of similar proofs. The decomposition predicts: the cost term $\log(\kappa_{\text{big}}/\kappa_{\text{small}})$ dominates because $\varphi > 0$ and $\varepsilon_{\text{small}}$ is small. This is the specialization-friendly regime.

**Library-heavy formalization** ($M_{\text{eff}} \approx 8$–$15$). These lemmas require knowing which specific Mathlib lemma to invoke—a retrieval problem embedded in a proof-construction

Table 1: Three-regime taxonomy for formal theorem proving in Lean 4. Columns marked [E] contain empirically estimated values reflecting domain judgment; these would differ for other task domains. Columns marked [F] are framework-derived: they follow from the decomposition structure given the mode count. A practitioner in a different domain must estimate their own mode counts via the protocol in Section 9.

| **Regime** | $M_{\text{eff}}$ **[E]** | **Transfer from coding [E]** | **Recommended model class [F]** | **Likely bottleneck [F]** | **Recommended intervention [F]** |
|---|---|---|---|---|---|
| Routine tactic application | 3–5 | Strong | Small fine-tuned specialist | Cost ($\kappa$) | Cheaper model, more attempts |
| Library-heavy formalization | 8–15 | Partial | Medium model + retrieval augmentation | Routing ($\varepsilon$) | Better retrieval, mode-specific routing |
| Creative theorem search | 20+ | Weak | Frontier generalist | Competence ($\varphi$) or information ($G$) | Better training data, broader pre-training |

problem. The mode space is moderately wide: multiple viable proof strategies exist, and the key differentiator is whether the model can locate the right library fact. The decomposition predicts: the routing term $\varepsilon$ is the primary bottleneck. A specialist may have low $\kappa$ but high $\varepsilon$ if its training did not cover the relevant library corner. This is the contested regime where the recommendation is conditional.

**Creative theorem search** ($M_{\text{eff}} \approx 20+$). These problems require multi-step combinations of tactics, novel lemma inventions, and proof-state abstractions not well represented in any existing training corpus. The decomposition predicts: either $\varphi$ or $G$ is the bottleneck, because even a well-routed model needs to *discover* strategies not in its training distribution. The cost advantage of a specialist is eroded by the routing penalty $\varepsilon$.

### 8.2 Worked examples

**Example 1 (Undergraduate Lean Agent — Specialist Dominates)** *Team A builds a Lean 4 agent for undergraduate-level theorem proving. They consider DeepSeek-Prover-V1.5 (7B specialist, $\kappa_A = \$0.001$/call) versus Claude Opus 4 (frontier generalist, $\kappa_B = \$0.03$/call) under a monthly budget of \$500.*

***Decomposition estimation.*** *From a 50-problem development set (satisfying the screening heuristic: $\kappa$ ratio $= 30\times$, narrow domain, $> 50$ problems):*

| | *Model* | $\log\frac{\kappa_0}{\kappa}$ | $\varphi$ | $G$ | $-\varepsilon$ | $\Delta$ |
|---|---|---|---|---|---|---|
| *Routine ($M_{\text{eff}} \approx 3$)* | *DeepSeek-7B* | +1.70 | +0.90 | +0.80 | −0.10 | +3.30 |
| | *Claude Opus* | −1.70 | 0.00 | +0.90 | −0.55 | −1.35 |

Illustrative values under model parameters from published benchmarks and structural predictions of the framework.

***Verdict: deploy the specialist.*** *The specialist advantage is* $\Delta_A - \Delta_B = 4.65$ *nats, corresponding to* $\exp(4.65) \approx 105\times$ *speedup under equalized cost. All four terms favor the specialist. The decomposition shows* why*: cost (*$+3.40$ *nats from* $30\times$ *cheaper price), competence (*$+0.90$ *nats from faster convergence on induction/simp patterns), and routing (*$\varepsilon_A = 0.10 \ll \varepsilon_B = 0.55$*). Information is approximately equal. Even if the competence advantage* $\varphi$ *were zero, the cost advantage alone exceeds any plausible combination of routing penalty and information gap in this narrow-mode regime.*

***What benchmark racing would say.*** *"DeepSeek wins 88% of problems." Correct but uninformative: it does not explain which lever drives the advantage, whether the advantage persists under different cost assumptions, or what would need to change for the recommendation to flip.*

**Example 2 (Research-Level Lean Agent — Conditional Verdict)** *Team B builds a Lean 4 agent for research-level formalization: library-heavy proofs requiring knowledge of specific Mathlib4 lemmas, with 8–15 effective modes.*

***Decomposition estimation.*** *From a 100-problem development set:*

| | *Model* | $\log\frac{\kappa_0}{\kappa}$ | $\varphi$ | $G$ | $-\varepsilon$ | $\Delta$ |
|---|---|---|---|---|---|---|
| *Library* ($M_{\text{eff}} \approx 12$) | *DeepSeek-7B* | $+1.70$ | $+0.10$ | $+1.60$ | $-0.85$ | $+2.55$ |
| | *Claude Opus* | $-1.70$ | $0.00$ | $+1.80$ | $-0.35$ | $-0.25$ |

*The specialist advantage is* $\Delta_A - \Delta_B = 2.80$ *nats. But this headline number conceals a fragile structure.*

***Sensitivity.*** *The specialist's routing mismatch* $\varepsilon_A = 0.85$ *nats is large: it wastes attempts on tactic families that do not match the library-heavy mode structure. If the specialist's routing could be improved to* $\varepsilon_A = 0.45$ *(via retrieval augmentation), the advantage grows to* $3.20$ *nats—robust. But on the roughly 20% of library-heavy problems where only Claude succeeds (because the specialist has never seen the required Mathlib lemma), no amount of cost advantage helps. The expected advantage, weighting by coverage, may be near zero or negative.*

***Verdict: conditional.*** *The framework identifies this as a close call and provides specific conditions under which the recommendation flips: (i) if the team can reduce* $\varepsilon_A$ *via retrieval augmentation, the specialist dominates; (ii) if frontier-only coverage matters, the generalist is safer; (iii) if* $\hat{c}$ *is uncertain, the team should run the full pilot. The framework's prescription is "invest in routing improvement before committing to the specialist"—qualitatively different from benchmark racing's "just try both."*

***What benchmark racing would say.*** *"DeepSeek wins 65% of problems overall." The decomposition reveals this conflates the 80% where both succeed (specialist wins most by cost) and the 20% where only the generalist succeeds.*

### 8.3 Coding-to-math transfer

The framework provides a nuanced answer to whether training on code corpora helps with formal theorem proving.

**Narrow-mode regime (routine tactic application).** The dominant modes—pattern matching on goal structure, symbolic rewriting, search over a small tactic vocabulary—overlap substantially with code-generation modes. Post-training equivalence predicts that coding-task fine-tuning transfers well.

**Wide-mode regime (creative theorem search).** The dominant modes—lemma invention, proof-state abstraction, library navigation across deep dependency chains—are not represented in code corpora. The transfer analysis predicts: source-task fine-tuning on code is insufficient because the target has modes not present in the source. Broader mathematical pre-training matters more.

**The opposing conclusions.** The same framework yields: *code training helps* for routine tactic application and *code training does not help* for creative theorem search. This reflects the structural difference in mode overlap between coding and proving across narrow versus wide mode regimes. A benchmark that averages across regimes would report "partial transfer," masking the fact that transfer is strong where it does not matter much and weak where it matters most.

### 8.4 Four failure modes

The four-term decomposition provides a diagnostic framework for understanding why a math agent is slow.

1. **Missing structure (low $G$).** The agent's proof traces show that knowing the problem identity does not help predict which strategy works. The model lacks the representations to extract information from the proof state. *Intervention:* better training data covering the missing modes.
2. **Bad routing (high $\varepsilon$).** The agent attempts tactics in the wrong order or wastes attempts on strategies that cannot work for the given goal structure. *Intervention:* improve retrieval or add mode-specific routing.
3. **Expensive nodes (high $\kappa$).** Each attempt costs too much, consuming budget before the model can explore enough strategies. *Intervention:* switch to a cheaper model or reduce per-step cost via distillation.
4. **Bad graph shape (critical-path bottleneck).** The orchestration forces sequential steps when parallel branching would be more efficient. *Intervention:* restructure the workflow.

## 9 Measurement Protocol and Empirical Program

The decomposition is useful only to the extent that $\kappa$, $\varphi$, $G$, $\varepsilon$, and $M_{\text{eff}}$ can be estimated from data. A domain-agnostic recipe, concrete estimators, and a pilot study design follow.

### 9.1 Domain-agnostic mode-discovery recipe

Before any domain-specific measurement, the builder must identify the latent modes. The recipe has four steps, of which only Step 2 requires domain knowledge.

1. *Collect successful solution traces* from the target task. Run candidate models on a development set. Pool all successful traces.

2. *Define a trace-level similarity metric* appropriate to the domain. This is the one domain-specific step: theorem proving uses edit distance on tactic sequences; code generation uses AST diff; text generation uses embedding cosine similarity; retrieval-augmented systems use tool-call sequence alignment.
3. *Cluster with stability selection.* Apply hierarchical clustering at multiple thresholds. Retain the mode count that is stable across bootstrap resamples. Report the stability range.
4. *Validate predictiveness.* Check that mode labels predict differential model performance: success rates must differ meaningfully across clusters for at least one model. If mode labels do not predict which model succeeds, the discovered modes are not design-relevant.

Steps 1, 3, and 4 are domain-agnostic. Step 2 requires domain knowledge but is a well-posed question.

### 9.2 Screening heuristic

Before investing in the full measurement protocol, practitioners should apply a cheap screening test.

| **Condition** | **Action** | **Rationale** |
|---|---|---|
| $\kappa$ ratio $> 50\times$ *and* narrow domain | Deploy specialist, no pilot needed | Cost term ($> 3.9$ nats) dominates all plausible $\varphi, G, \varepsilon$ |
| $\kappa$ ratio $< 3\times$ | Benchmark-race with small comparison set | Cost term ($< 1.1$ nats) is comparable to estimation noise |
| Fewer than $\sim 50$ verifiable problems | Do not run pilot | Mode labeling requires sufficient within-cluster samples |
| $\kappa$ ratio 3–50$\times$, moderate complexity, $\geq 50$ problems | Run the full pilot | Decomposition's marginal value over benchmark racing is concentrated here |

This converts the measurement protocol from "always do this" to "do this when it is worth doing"—consistent with a framework that includes cost in its own decomposition.

### 9.3 Estimator summary

All estimators require the mode-labeling step above. The following table summarizes.

| Quantity | Observable / Estimator | Main Error Source |
|---|---|---|
| $\kappa(M)$ | Per-step cost from API pricing: $\hat{\kappa}$ = price/1K tokens × tokens/call; or latency. | Variance in tokens/call. Use median. |
| $\varphi(M_0, M)$ | Within-mode competence gap: $\hat{\varphi} = \sum_s \hat{\pi}(s) \cdot \log(\hat{t}_0(M_0, s)/\hat{t}_0(M, s))$, where $\hat{t}_0$ is median solve time per mode. | Small sample per mode. Bootstrap CIs. |
| $G(M)$ | $\hat{G} = H(\hat{\pi}) - \frac{1}{N}\sum_i H(\hat{\pi}^M_{D_i})$. Under verifier equalization: check $\lvert\hat{G}(M_1) - \hat{G}(M_2)\rvert < 0.5$ nats; if so, use the average. | Entropy estimation bias. Miller–Madow correction: $\hat{H}_{\mathrm{MM}} = \hat{H} + \frac{M_n - 1}{2N}$. |
| $\varepsilon(M)$ | $\hat{\varepsilon} = \frac{1}{N}\sum_i \widehat{\mathrm{KL}}(\hat{\pi}^M_{D_i} \Vert \hat{q}^M_i)$. Frequency proxy if model does not output mode probabilities. | KL plug-in estimator biased upward. Laplace smoothing. |
| $M_{\mathrm{eff}}$ | $\overline{M}_{\mathrm{eff}} = \exp\Big(\frac{1}{N}\sum_i H(\hat{\pi}^{M_{\mathrm{ref}}}_{D_i})\Big)$, where $M_{\mathrm{ref}}$ is the cheapest candidate model. | Mode-labeling uncertainty propagates directly. |

### 9.4 Pilot study design

We specify a pilot for comparing DeepSeek-Prover-V1.5 (7B specialist) against Claude Opus 4 (frontier generalist) on 300 Mathlib4 lemmas.

**Benchmark construction.** Extract candidate lemmas from Mathlib4 (commit pinned after DeepSeek-Prover training cutoff). Classify into three strata by top-level tactic diversity: routine ($\leq 2$ tactic families), library-heavy (specific Mathlib lemma invocations), and creative ($\geq 3$ tactic families, no single tactic $> 50\%$ of steps). Sample 100 per stratum; split 50/50 into development and test sets.

**Budget equalization.** Equalize total API cost, not number of attempts. At $\kappa_A =$ \$0.001/call and $\kappa_B =$ \$0.03/call with \$0.064/problem: DeepSeek gets 64 attempts per problem, Claude gets $\approx 56$.

**Mode labeling.** Pool successful proof traces from both models on the development set. Extract top-level tactic sequences; compute pairwise normalized edit distance; hierarchical agglomerative clustering with average linkage. Sweep distance threshold and retain the $M_{\mathrm{eff}}$ stable across bootstrap resamples. Fix labels before computing any model-dependent quantities.

**Five testable predictions.** The pilot tests five qualitative predictions: (P1) DeepSeek has $\sim 30\times$ lower per-call cost; (P2) DeepSeek has better competence on routine problems ($\varphi > 0$); (P3) Claude has higher information gain on creative problems; (P4) DeepSeek

has lower routing mismatch on routine problems; (P5) a crossover exists: specialist wins low-$M_{\text{eff}}$, generalist wins high-$M_{\text{eff}}$.

**Power analysis.** Under log-normal certified times, a two-sample $t$-test on log-speedup differences with effect size $d = 0.5$, $\alpha = 0.05$, and 80% power requires $\sim 64$ problems per group. The 150-problem test set (50 per stratum) provides adequate power for stratum-level comparisons.

**Honest assessment criterion.** If prediction accuracy in the contested regime (library-heavy) is near 50% (within $[45\%, 55\%]$), the decomposition adds nothing beyond a cost-based rule of thumb, and the paper reports this as a negative result.

## 10 Discussion and Conclusion

### 10.1 Limitations

The framework rests on the packed latent-mode family. The assumption that certified time scales as $t_0(s)/q(s)$—that doubling the compute share halves the expected time—is a geometric-trials model. The framework does not model correlated strategies, where success on one approach provides partial information for another. The mode-labeling step is a clustering judgment, not a mathematical object: different clustering thresholds yield different mode counts, and the decomposition values change accordingly.

The framework assumes an external verifier. Tasks without binary or continuous verification—open-ended generation, creative writing, policy analysis—fall outside its scope. The framework addresses this limitation by not pretending to cover these tasks.

The four-term decomposition is exact only in the packed-family regime. Outside it, the approximation guarantee (Proposition 25) provides a bound with remainder $O(\delta \log M_n + \eta)$. For tasks with many soft modes or strongly nonlinear time scaling, the remainder may dominate.

The measurement protocol requires a pilot study. The pilot is cheap (estimated \$725–\$775 for the Lean study) but not free. The screening heuristic (Section 9.2) mitigates this by identifying when the pilot is worth running.

### 10.2 Open questions

1. **Joint optimization.** The framework solves the model-choice sub-problem of the master design program (2). Full joint optimization over all six coordinates—model, routing, tools, memory, context, parallelism—remains open.
2. **Concavity of mixed-source objective.** The mixed-source post-training objective $\Delta_{\alpha \to B}(m)$ involves a mutual-information term (concave in $\alpha$) and a mismatch term (not generically convex). Whether the combined objective is concave under natural structural conditions is unknown.
3. **Dynamic mode discovery.** The current framework assumes modes are fixed. An adaptive version that discovers and refines modes during deployment—updating the decomposition as the system learns—would extend the framework to non-stationary task distributions.

4. **Estimation of $\mathrm{I}(S_B; Z_A)$.** The transfer characterization (Proposition 14) involves the mutual information between a target's latent modes and a source's learned representation. Direct estimation requires model internals or representation probing and remains an open problem.
5. **Multi-agent coordination cost.** The multi-agent threshold (Theorem 29) treats $c_{\mathrm{coord}}(K)$ as exogenous. Deriving $c_{\mathrm{coord}}$ from the structure of the coordination protocol is an open problem connected to Radner's [31] team theory.

### 10.3 What the framework gives and does not give

The framework gives the builder four things. First, a decomposable objective: instead of a single opaque comparison, the builder sees four channels, each independently improvable. Second, a crossover formula: the specialist-versus-generalist boundary is computable, not guessable. Third, a transfer characterization: the builder knows when source-task training helps, when to retrain the router, and when broader pre-training is needed. Fourth, a measurement protocol: every quantity in the decomposition is estimable from a structured pilot.

The framework does not give the builder a solved optimization problem. It does not choose the orchestration architecture, the tool library, the memory system, or the parallelism degree. It does not replace engineering judgment. It replaces *unstructured* engineering judgment with *structured* engineering judgment: the builder still decides, but now knows which channel to invest in and how much each channel contributes.

The gap between the framework and a full agent-design compiler is large. The framework occupies a middle ground between theory that is too abstract to be actionable and engineering that is too ad hoc to be systematic. Whether that middle ground is useful is ultimately an empirical question—one the pilot study is designed to answer.

## Appendix A. Continuous-Reward Extension

The main body restricts to binary verification: a task instance either produces a verified solution ($V_n = 1$) or does not. This appendix generalizes the framework to continuous reward signals $R_n : \mathcal{X} \times \mathcal{Y} \to [0, R_{\max}]$, where $R_n(x, y)$ measures the quality of output $y$ on input $x$. The generalization is structurally parallel to the binary case; the algebraic decomposition carries through with three specific modifications.

### A.1 Setup

**Definition 31 (Continuous-reward task family)** *A* continuous-reward task family *is a tuple $(\mathcal{X}_n, \mathcal{Y}_n, R_n, \mathcal{B}_n, P_n)$ where $\mathcal{X}_n$ is the input space, $\mathcal{Y}_n$ is the output space, $R_n : \mathcal{X}_n \times \mathcal{Y}_n \to [0, R_{\max}]$ is the reward function (measurable, bounded), $\mathcal{B}_n$ is the budget, and $P_n$ is the task distribution over $\mathcal{X}_n$. The binary case is recovered by setting $R_n = \mathbf{1}\{V_n = 1\}$ with $R_{\max} = 1$.*

### A.2 Three modifications

**Modification 1: Success probability → expected reward per mode.** The mode-conditional log-time $\log t_0(M, s) = \log(1/p_s(M))$ generalizes to the mode-conditional log-

reward-rate: $\log t_0^R(M,s) := -\log r_s(M)$, where $r_s(M) := \mathbb{E}[R_n(X,Y) \mid S = s, \text{model } M]$. When $R_n$ is binary, $r_s = p_s$ and the expressions coincide.

**Modification 2: Certified time → certified reward-time.** The certified time $T_q^M(s)$ is replaced by $T_q^{R,M}(s) := \kappa(M) \cdot c_{\text{sim}}/(q(s) \cdot r_s(M))$. In log-space: $\log T_q^{R,M}(s) = \log \kappa(M) + \log c_{\text{sim}} + \log t_0^R(M,s) + \log(1/q(s))$—the same additive structure as the binary case.

**Modification 3: Posterior via reward observations.** The posterior is updated via Bayes' rule on reward observations: $\pi_D^{R,M}(s) \propto \pi(s) \cdot \prod_{j \in D} p(R_j \mid S = s, M)$. The information gain becomes $G^R(M) = I_\pi(S : D_M^R)$.

### A.3 Generalized decomposition theorem

**Theorem 32 (Four-term decomposition under continuous rewards)** *Under the packed-family assumption with continuous rewards (Modifications 1–3), the expected certified log-reward-time decomposes as:*

$$\Delta^R := \mathbb{E}\big[\log T_\pi^{R,M_0}(S) - \log T_{q_D}^{R,M}(S)\big] = \log \frac{\kappa(M_0)}{\kappa(M)} + \varphi^R(M_0, M) + G^R(M) - \varepsilon^R(M),$$

*where $\varphi^R(M_0, M) := \mathbb{E}_\pi[\log(r_S(M)/r_S(M_0))]$ is the competence advantage, $G^R(M) := I_\pi(S : D_M^R)$ is the information gain from reward observations, and $\varepsilon^R(M) := \mathbb{E}_D[\mathrm{KL}(\pi_D^{R,M} \| q_D)]$ is the routing mismatch.*

**Proof** The proof is identical in structure to the binary case (Theorem 21). The additive log-structure of certified reward-time ensures the cross-entropy identity applies. The tower property $\mathbb{E}_D[\mathbb{E}_{\pi_D^{R,M}}[\log t_0^R(M,S)]] = \mathbb{E}_\pi[\log t_0^R(M,S)]$ holds by the same argument as in the binary case. The four terms emerge identically. ■

### A.4 Binary recovery

Setting $R_n = \mathbf{1}\{V_n = 1\}$: $r_s(M) = p_s(M)$, $t_0^R = t_0$, $T_q^{R,M} = T_q^M$, $\pi_D^{R,M} = \pi_D^M$, and $\Delta^R = \Delta$. All terms reduce exactly to their binary counterparts.

### A.5 Connection to optimal stopping

The routing problem in the continuous-reward case generalizes to a stopping/multi-armed-bandit structure. This connects to Weitzman's [42] Pandora's Box (each mode is a "box" with opening cost proportional to $\kappa(M)/q(s)$ and expected reward $r_s(M)$; the routing mismatch measures the gap between the builder's allocation and the reservation-value ranking) and to Zilberstein's [47] anytime-algorithm framework (the reward function is the natural quality measure and the four-term decomposition tells the builder why the system's reward-rate is what it is). The framework identifies the stopping/routing structure and decomposes the performance gap into four channels. It does not prove a new optimal stopping theorem: the inner routing problem is known, and the outer model-choice problem is the contribution of the four-term decomposition.

## Appendix B. Proofs of Supplementary Results

### B.1 Full proof of Proposition 17 (orchestration equivalence bounds)

**Proof** *Step 0: Pinsker bound.* By Pinsker's inequality applied pointwise: $\delta_D := \|\pi_D^A - \pi_D^B\|_1 \leq \sqrt{2\,\mathrm{KL}(\pi_D^A \| \pi_D^B)}$. Taking expectations and applying Jensen (concavity of $\sqrt{\cdot}$): $\bar{\delta} := \mathbb{E}_D[\delta_D] \leq \sqrt{2\alpha}$.

*Proof of (a).* Recall $G_A - G_B = \mathbb{E}_D[H(\pi_D^B)] - \mathbb{E}_D[H(\pi_D^A)]$. By Audenaert's entropy continuity applied pointwise: $|H(\pi_D^A) - H(\pi_D^B)| \leq \delta_D \log(M_n/\delta_D)$. Taking expectations: $|G_A - G_B| \leq \mathbb{E}_D[\delta_D \log(M_n/\delta_D)]$. Since $f(t) = t\log(M_n/t)$ is concave on $(0, M_n]$, Jensen gives $\mathbb{E}_D[f(\delta_D)] \leq f(\bar{\delta}) \leq \sqrt{2\alpha}\log(M_n/\sqrt{2\alpha})$.

*Proof of (b).* Use the KL-difference decomposition: $\mathrm{KL}(p\|q) - \mathrm{KL}(r\|q) = [H(r) - H(p)] + \sum_s [p(s) - r(s)]\log(1/q(s))$. Apply with $p = \pi_D^A$, $r = \pi_D^B$, take expectations. Bound the first term by part (a) and the second by $\bar{\delta} \cdot \log(1/q_{\min}) \leq \sqrt{2\alpha}\log(1/q_{\min})$.

*Proof of (c).* If $q_{\min} \geq 1/M_n$, then $\log(1/q_{\min}) \leq \log M_n$. Parts (a) and (b) are both $O(\sqrt{\alpha}\log M_n)$. ■

### B.2 Full proof of the alignment-certified upper envelope (Theorem 24)

See Section 6, §6.3 for the expectation-level statement. The pointwise argument follows the structure in the main text:

$$\overline{\mathrm{Sp}}(D_n) \leq \mathrm{I}_{\mathrm{poly}}(H_n : D_n) + \rho_D + \kappa_U.$$

Full details are provided in the earlier presentation.

## Appendix C. Scaling Laws (Compressed)

The four-term decomposition induces scaling predictions for how $\Delta$ changes with model scale and data scale.

**Pre-saturation regime.** When $G(M) \ll H(\pi)$ (the verifier feedback does not yet resolve all modes), increasing data or model quality increases $G$ approximately linearly. In this regime, improving the information channel is the highest-leverage intervention.

**Post-saturation regime.** When $G(M) \approx H(\pi)$ (the verifier has identified the dominant mode), further improvements come only from reducing $\kappa$ or $\varepsilon$. This is the regime where specialization dominates: the system already knows what to do, and the question is how cheaply and accurately it can execute.

**Crossover threshold scaling.** The crossover threshold $M_{\mathrm{eff}}^\star = \exp((\log(\kappa_{\mathrm{big}}/\kappa_{\mathrm{small}}) + \hat{\varphi})/\hat{c})$ grows exponentially with the cost ratio and competence advantage, and shrinks with richer verifier feedback ($\hat{c}$ larger). As frontier models become cheaper (compressing $\kappa_{\mathrm{big}}/\kappa_{\mathrm{small}}$), the specialization-viable region shrinks. This predicts that the value of task-specific fine-tuning decreases over time as inference costs fall—a testable prediction.

# Appendix D. Notation and Assumption Audit

## D.1 Notation reference

| Symbol | Definition |
|---|---|
| $\mathfrak{T} = (\mathcal{X}_n, \mathcal{Y}_n, V_n, \mathcal{B}_n)$ | Budgeted transductive task at scale $n$ |
| $\overline{T}_A(n, \delta)$ | Certified time-to-solution at confidence $1 - \delta$ |
| $\Delta$ | Certified log-speedup: $\log \overline{T}_{A_0} - \log \overline{T}_A$ |
| $S \in [M_n]$ | Latent mode variable |
| $\pi, \pi_D$ | Prior and posterior over modes |
| $q, q_D$ | Baseline and deployed routing allocations |
| $\kappa(M)$ | Per-step cost of model $M$ |
| $t_0(M, s)$ | Within-mode certified scale for model $M$ on mode $s$ |
| $\varphi(M_0, M)$ | $\mathbb{E}_\pi[\log(t_0(M_0, S)/t_0(M, S))]$: competence advantage |
| $G(M)$ | $\mathrm{I}_\pi(S : D_M)$: information generation quality |
| $\varepsilon(M)$ | $\mathbb{E}_D[\mathrm{KL}(\pi_D^M \| q_D^M)]$: routing mismatch |
| $M_{\text{eff}}$ | $\exp(H(\pi_D))$: entropy-effective mode count |
| $M_{\text{eff}}^\star$ | Crossover threshold: specialist dominates below, generalist above |
| $d = (M, \pi, \Omega, W, K, p)$ | Design tuple |
| $\alpha$ | Orchestration-equivalence parameter: $\mathbb{E}_D[\mathrm{KL}(\pi_D^A \| \pi_D^B)]$ |
| $\eta_{A \to B}$ | Post-training efficiency: transfer ratio from family $A$ to $B$ |
| $c_{\text{coord}}(K)$ | Coordination cost for $K$-agent team |
| $\phi_{\text{spec}}$ | $\sum_s \pi(s) \log(t_0(M_g, s)/t_0(M_{\text{team}(s)}, s))$: specialization gain |

## D.2 Assumption audit

The following table traces which assumptions each result requires.

| Result | Assumptions Required |
|---|---|
| RI Identity (Thm. 19) | Packed family (Asm. 6), prior-matched baseline |
| Four-term decomposition (Thm. 21) | Model-dependent packed family (Asm. 20) |
| Upper envelope (Thm. 24) | A1–A7 (Asm. 22), sufficient-statistic condition (Asm. 23) |
| Approximation guarantee (Prop. 25) | Relaxed packed family ($\delta$-soft modes, $\eta$-approximate log-linearity) |
| Crossover formula (Cor. 26) | Model-dependent packed family |
| Crossover threshold (16) | Additionally (S1)–(S3): verifier equalization, linear mismatch scaling, constant competence |
| Verifier equalization (Prop. 27) | Model-dependent packed family, small residual posterior entropy |
| Multi-agent threshold (Thm. 29) | Multi-agent packed family (Asm. 28), known $\gamma$ |
| Multi-agent corollary (Cor. 30) | Additionally: verifier equalization with bound $\eta$ |
| Orchestration equivalence (Prop. 17) | $\alpha$-orchestration equivalence (Def. 16), common mode bank |
| Post-training equivalence (Thm. 13) | Packed family, shared latent space |
| Transfer characterization (Prop. 14) | Target admits packed-family representation |
| Continuous-reward extension (Thm. 32) | Packed family with Modifications 1–3 |

## References


[1] A. Achille and S. Soatto. AI agents as universal task solvers: It's all about time. *Preprint*, 2026.

[2] R. Agarwal, M. C. Machado, P. S. Castro, and M. G. Bellemare. Contrastive behavioral similarity embeddings for generalization in reinforcement learning. In *ICLR*, 2021.

[3] K. M. R. Audenaert. A sharp continuity estimate for the von Neumann entropy. *J. Phys. A: Math. Theor.*, 40(28):8127, 2007.

[4] P.-L. Bacon, J. Harb, and D. Precup. The option-critic architecture. In *AAAI*, 2017.

[5] N. Baram et al. Behavioral mode discovery via trajectory clustering. *Preprint*, 2024.

[6] M. Besta et al. Graph of thoughts: Solving elaborate problems with large language models. *Preprint*, 2023.

[7] J. R. Birge and F. Louveaux. *Introduction to Stochastic Programming*. Springer, 2nd edition, 2011.

[8] S. Borgeaud et al. Improving language models by retrieving from trillions of tokens. In *ICML*, 2022.

[9] M. Chen et al. Evaluating large language models trained on code. *Preprint*, 2021.

[10] H. Chernoff. Sequential design of experiments. *Ann. Math. Stat.*, 30(3):755–770, 1959.

[11] T. M. Cover and J. A. Thomas. *Elements of Information Theory.* Wiley, 2nd edition, 2006.

[12] I. Csiszár and J. Körner. *Information Theory: Coding Theorems for Discrete Memoryless Systems.* Cambridge University Press, 2nd edition, 2011.

[13] K. Ellis et al. DreamCoder: Learning to code by writing programs in your sleep. In *PLDI*, 2021.

[14] W. Fedus, B. Zoph, and N. Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 23(120):1–39, 2022.

[15] N. Ferns, P. Panangaden, and D. Precup. Metrics for finite Markov decision processes. In *UAI*, 2004.

[16] M. Feurer et al. Efficient and robust automated machine learning. In *NeurIPS*, 2015.

[17] C. P. Gomes and B. Selman. Algorithm portfolios. *Artif. Intell.*, 126(1–2):43–62, 2001.

[18] G. Grand et al. LILO: Learning interpretable libraries by compressing and documenting code. *Preprint*, 2023.

[19] E. A. Hansen and S. Zilberstein. Monitoring and control of anytime algorithms: A dynamic programming approach. *Artif. Intell.*, 126(1–2):139–157, 2001.

[20] B. A. Huberman, R. M. Lukose, and T. Hogg. An economics approach to hard computational problems. *Science*, 275(5296):51–54, 1997.

[21] F. Hutter, H. Hoos, and K. Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *LION*, 2011.

[22] M. I. Jordan and R. A. Jacobs. Hierarchical mixtures of experts and the EM algorithm. *Neural Comput.*, 6(2):181–214, 1994.

[23] J. L. Kelly. A new interpretation of information rate. *Bell Syst. Tech. J.*, 35(4):917–926, 1956.

[24] L. A. Levin. Universal sequential search problems. *Probl. Peredachi Inf.*, 9(3):115–116, 1973.

[25] L. A. Levin. Randomness conservation inequalities. *Inform. Control*, 61(1):15–37, 1984.

[26] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.

[27] Y. Li et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

[28] J. Marschak and R. Radner. *Economic Theory of Teams.* Yale University Press, 1972.

[29] M. Li and P. Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, 3rd edition, 2008.

[30] M. S. Pinsker. *Information and Information Stability of Random Variables and Processes*. Holden-Day, 1964. (Russian original 1960.)

[31] R. Radner. Team decision problems. *Ann. Math. Stat.*, 33(3):857–881, 1962.

[32] B. Ravindran and A. G. Barto. SMDP homomorphisms: An algebraic approach to abstraction in semi-Markov decision processes. In *IJCAI*, 2003.

[33] S. Russell and E. Wefald. *Do the Right Thing: Studies in Limited Rationality*. MIT Press, 1991.

[34] J. Schmidhuber. The speed prior: A new simplicity measure yielding near-optimal computable predictions. In *COLT*, 2002.

[35] N. Shazeer et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*, 2017.

[36] C. Snell et al. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *ICLR*, 2025.

[37] R. J. Solomonoff. A formal theory of inductive inference, Part I. *Inform. Control*, 7(1):1–22, 1964.

[38] R. S. Sutton, D. Precup, and S. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artif. Intell.*, 112(1–2):181–211, 1999.

[39] M. E. Taylor and P. Stone. Transfer learning for reinforcement learning domains: A survey. *JMLR*, 10:1633–1685, 2009.

[40] A. Wald. *Sequential Analysis*. Wiley, 1947.

[41] X. Wang et al. Self-consistency improves chain of thought reasoning in language models. In *ICLR*, 2023.

[42] M. L. Weitzman. Optimal search for the best alternative. *Econometrica*, 47(3):641–654, 1979.

[43] L. Xu, F. Hutter, H. H. Hoos, and K. Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *JAIR*, 32:565–606, 2008.

[44] S. Yao et al. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*, 2023.

[45] J. Zhang et al. AFlow: Automating agentic workflow generation. *Preprint*, 2024.

[46] K. Zheng et al. MiniF2F: A cross-system benchmark for formal Olympiad-level mathematics. In *ICLR*, 2022.

[47] S. Zilberstein. Using anytime algorithms in intelligent systems. *AI Magazine*, 17(3):73–83, 1996.